

AE4320 Assignment 2026: Multivariate Simplex Splines

This individual take-home assignment consists of 4 parts. The deliverable of the assignments are a report of **maximum 25 pages** containing the theory and results of the 4 parts, a set of Matlab or Python code files which were used to generate the results, **and all generative AI prompts that you used** to complete the assignment. A total of **100 points** can be obtained for the assignment. In **Appendix A**, a brief introduction to the measurement dataset and special validation dataset is given. In **Appendix B** details for the construction of the Kalman Filter are given. In **Appendix C**, a brief description of the supplied Matlab files is given.

Important: your report should have the same organization (and chapter numbering) as the assignment text!

Part 1: State & Parameter estimation with F-16 flight data (35 points)

You are asked first to implement a state estimation routine (Kalman Filter) to estimate a bias in the angle of attack measurements in the given dataset.

It is given that there is an unknown bias in the angle of attack (α_m) measurements caused by upwash. Also, a sensor glitch has been reported that may have corrupted part of the C_m measurements. The angle of sideslip (β_m) measurements and velocity (V_m) are not biased. Please see **Appendix B** for details on the construction of the Kalman Filter. **The report should contain the following items:**

1. Explain how you treated the given dataset, e.g. did you pre-filter the data, remove glitches, how did you define identification and validation datasets? **(6 points)**
2. Formulate the (trivial) system equations $\dot{x}_k = f(x_k, u_k, t)$ and the (non-trivial) output equations $z = h(x_k, u_k, t)$ given the information in **Appendix B**. **(3 points)**
3. Briefly motivate which type of Kalman filter should be used in this case; the linear Kalman Filter, the Extended Kalman Filter, or the Iterated Extended Kalman Filter. **(2 points)**
4. Construct your Kalman filter, and use it to estimate $C_{\alpha_{up}}$ (clearly present the numerical result!). **Important:** Use a fixed time step of $\Delta t = 0.01$ in your integrator/continuous-to-discrete time conversion operation, otherwise the input vector cannot be used correctly. Prove that your filter has converged. **(5 points)**
5. Reconstruct α_{true} using the estimated $C_{\alpha_{up}}$, and clearly show the difference with the measured α_m . **(2 points)**

Implement an ordinary least squares estimator for a standard polynomial model structure for the reconstructed F-16 flight data and apply it on the given dataset. The report/code should contain the following items:

6. Formulate a least squares estimator, and estimate the coefficients of your polynomial model. **(5 points)**

7. Show the influence of polynomial model order on the accuracy of fit. Do this for both the measurement dataset as well as the 'special validation dataset' that was included in the assignment. **(5 points)**
8. Show the results from the model validation; perform both a statistical and a model-error based validation. Do this for the measurement dataset (**not** the 'special validation dataset') that was included in the assignment. **(5 points)**
9. Provide the residual RMS of your selected polynomial model when validated using the special validation dataset that was included in the assignment. **Note** that the special validation dataset domain not necessarily overlaps the measurement data domain! **(2 points)**
10. Hand in the code used in this part in a separate, well documented file.

Part 2: Deriving a simplex polynomial (20 points)

You are asked to formulate and implement an estimator for the B-coefficients of a single simplex polynomial and use it to identify a model of the F-16 data using the reconstructed states:

1. Show how the basis functions of the simplex splines for general polynomial degrees are defined. Show how the barycentric coordinates are used in the basis functions. Show also how increasing the degree affects the structure of the basis polynomials. **(3 points)**
Hint: Don't forget the multinomial coefficient!
2. Clearly show how you placed the vertices and/or subdivided the data **(5 points)**.
3. Formulate a least squares estimator for the B-coefficients of a B-form polynomial, and use it to identify models for at least 6 different polynomial degrees based on the F-16 data. **(5 points)**
4. Evaluate the performance of your estimator using **both** a model residual and statistical model quality analysis. Do this for the measurement dataset (**not** the special validation dataset). Clearly show how the polynomial degree affects model quality. **(5 points)**
5. Provide the residual RMS of your selected simplex polynomial model when validated using the special validation dataset that was included in the assignment. **Note** that the special validation dataset domain not necessarily overlaps the measurement data domain! **(2 points)**
6. Hand in the code used in this part in a separate, well documented file.

Part 3: System Identification with Simplex Splines (35 points)

You are asked to formulate and implement an estimator for the B-coefficients of a complete simplex spline and use it to identify a high fidelity model of the reconstructed F-16 data:

1. Design (graphically) a complete system identification algorithm that combines state estimation techniques with an estimator for the B-coefficients of the multivariate simplex splines. **(3 points)**
2. Clearly show how you treated the data (e.g. pre-filtered, subdivided, etc) for this case. What patterns in the data motivated these actions? Clearly report your findings. **(2 points)**
3. Formulate the Smoothness Matrix for 0^{th} order continuity for your spline model. Prove that your model really has 0^{th} order continuity. **(5 points)**
Bonus: formulate the 1^{st} and higher continuity order smoothness matrix for your spline model, and again prove that higher order continuity is present. **(4 bonus points)**
4. Formulate a least squares estimator **(2 points)**, and a null-space projection based estimator **(3 points)** for the multivariate simplex splines using a triangulation consisting of at least 2 simplices. **(max 5 points)**
5. Implement the system identification framework for multivariate splines and use it to identify a multivariate spline based aerodynamic model for the F-16 given the flight data. Demonstrate the effect on the model quality of changing the polynomial degree, continuity order, and number of simplices. **(8 points)**
6. Validate your aerodynamic model using model residual and statistical model quality analysis. Provide the residual RMS of your simplex spline model. Do this using the validation dataset you derived from the measurement dataset (so **not** the special validation dataset). Clearly report your findings in the report **(8 points)**
7. Provide the residual RMS of your selected simplex spline model when validated on the special validation dataset that was included in the assignment. **Note** that the special validation dataset domain not necessarily overlaps the measurement data domain. **(4 points)**
8. Hand in the code used in this part in a separate, well documented file.

Part 4: Conclusion (10 points)

You are asked to write a brief conclusion (max 1 page) for your report which should contain the following items:

1. Compare the numerical results from the modelling with the standard polynomial model, the simplex polynomial model and the simplex spline model with each other. Provide a clear numerical comparison in a table in which the most important validation metrics are presented. **(4 points)**
2. Create a table with the strengths and weakness of each approach. Include a computational complexity analysis of the algorithms in your conclusion. **(6 points)**

Appendix A: Introduction to the F16 dataset.

Measurement Dataset

The file 'F16traindata_CMabV_2025.mat' contains the data in the Matlab MAT format , 'F16traindata_CMabV_2025.csv' contains the data in CSV format, which is more useful when using e.g. Python. The data is loaded from disk with Matlab using the command:

```
load('F16traindata_CMabV_2025.mat', 'Cm', 'Z_k', 'U_k');
```

The vector 'Cm' contains the measurement data of the pitching moment coefficient: $C_m \in \mathbb{R}^{N \times 1}$ with 'N' the total number of measurements. The matrix 'Z_k' contains the measurement locations (states), where Z_k contains the following data: $Z_k = [\alpha_m \quad \beta_m \quad V_m] \in \mathbb{R}^{N \times 3}$. The matrix 'U_k' contains perfect linear accelerometer measurements that can be used in your Kalman filter. It contains the following data: $U_k = [\dot{u} \quad \dot{v} \quad \dot{w}] \in \mathbb{R}^{N \times 3}$, with $\dot{u}, \dot{v}, \dot{w}$ the perfect (no biases, noise-free) time derivatives of the body velocities u, v, w . **Please do not confuse the longitudinal velocity 'u' with the system input U_k !**

When plotting the data, which can be done with the supplied file 'F16_PlotData.m' you will find that α_m, β_m, V_m are contaminated with zero mean white noise. In addition α_m is biased due to upwash at the location of the angle of attack vane, see **Appendix B**. The measurements made on Cm are also contaminated with noise.

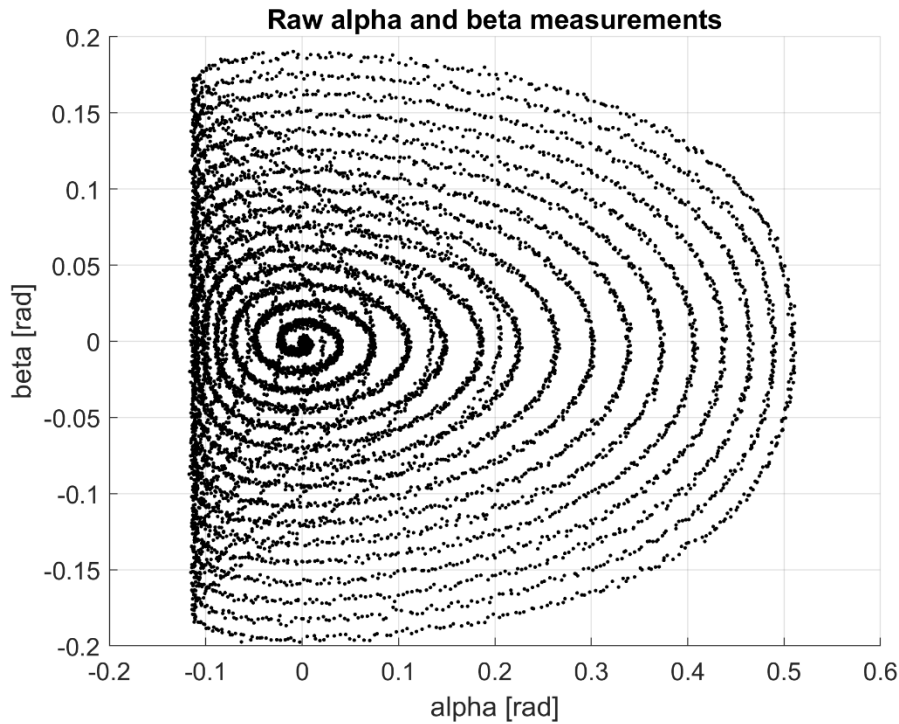


Figure 1: Locations in the angle of attack and angle of sideslip domain at which measurements of Cm are available.

Special validation dataset

This year, we have a **special validation dataset** which contains noise-free data on a very few select alpha,beta points. **These points do not need to be processed with your Kalman filter!**

The file 'F16validationdata_2025.mat' contains the **special validation dataset** in the Matlab MAT format , 'F16validationdata_2025.csv' contains the data in CSV format, which is more useful when using Python. The special validation dataset is loaded from disk with Matlab using the command:

```
load('F16validationdata_2025.mat', 'Cm_val', 'alpha_val', 'beta_val');
```

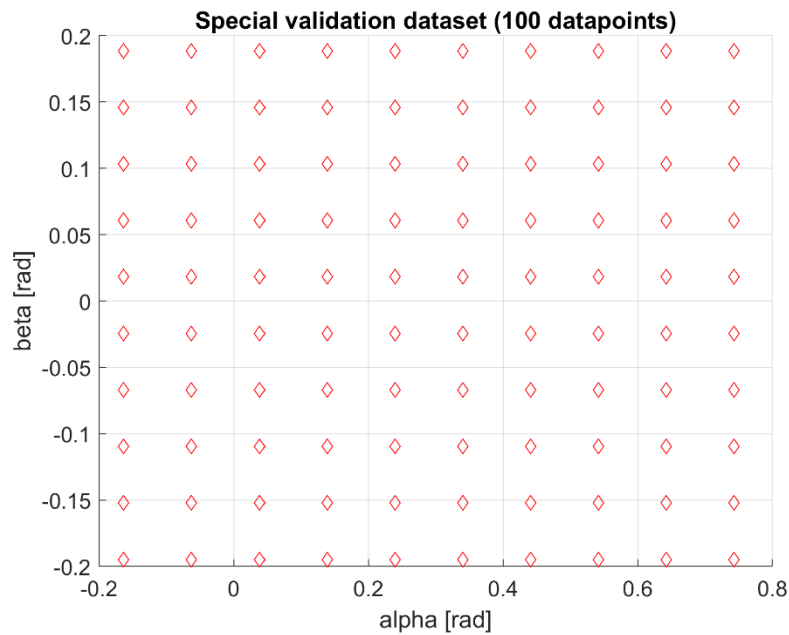


Figure 2: special validation dataset consisting of 100 points laid out on rectangular grid

Appendix B: Constructing the Kalman filter

In the construction of your Kalman filter, use the following state vector:

$$x_k = \begin{bmatrix} u & v & w & C_{\alpha_{up}} \end{bmatrix}^T$$

As measurement vector use:

$$z_k = \begin{bmatrix} \alpha_m & \beta_m & V_m \end{bmatrix}^T$$

As input vector use the perfect (**unbiased, noise-free**) linear accelerometer values, which are sampled with $\Delta t = 0.01$:

$$u_k = \begin{bmatrix} \dot{u} & \dot{v} & \dot{w} \end{bmatrix}$$

It is given that there is an unknown bias in the angle of attack (α_m) measurements caused by upwash at the location of the angle of attack vane. The angle of sideslip (β_m) measurements and velocity (V_m) are not biased. It is given that the **measured** angle of attack (α_m), angle of sideslip (β_m), and velocity (V_m) are related to their **true** values ($\alpha_{true}, \beta_{true}, V_{true}$) by the following formulas:

$$\alpha_m = \alpha_{true}(1 + C_{\alpha_{up}}) + v_\alpha$$

$$\beta_m = \beta_{true} + v_\beta$$

$$V_m = V_{true} + v_V$$

With $C_{\alpha_{up}}$ an unknown upwash coefficient that must be estimated, and with v_α , v_β , and v_V white noise sequences. The true angle of attack, angle of sideslip, and velocity are related to the system states as follows:

$$\begin{aligned} \alpha_{true} &= \tan^{-1} \left(\frac{w}{u} \right) \\ \beta_{true} &= \tan^{-1} \left(\frac{v}{\sqrt{u^2 + w^2}} \right) \\ V_{true} &= \sqrt{u^2 + v^2 + w^2} \end{aligned}$$

Substituting these expressions in the equations for α_m , β_m , and V_m will give you the observation equations $h(x, u, t) + \begin{bmatrix} v_\alpha & v_\beta & v_V \end{bmatrix}^T$.

You can use the following statistics for the process noise:

$$E\{\underline{w}(t_i)\} = 0; \quad E\{\underline{w}(t_i)\underline{w}^T(t_j)\} = Q\delta_{ij}; \quad Q = \text{diag}(\sigma_{w_u}^2, \sigma_{w_v}^2, \sigma_{w_w}^2, \sigma_{w_c}^2)$$

$$\sigma_{w_u} = 1e-3, \sigma_{w_v} = 1e-3, \sigma_{w_w} = 1e-3, \sigma_{w_c} = 0$$

And the following sensor noise statistics:

$$E\{\underline{v}(t_i)\} = 0; \quad E\{\underline{v}(t_i)\underline{v}^T(t_j)\} = R\delta_{ij}; \quad R = \text{diag}(\sigma_{v_\alpha}^2, \sigma_{v_\beta}^2, \sigma_{v_V}^2)$$

$$\sigma_{v_\alpha} = 1.5e-3, \sigma_{v_\beta} = 1.5e-3, \sigma_{v_V} = 1.0e0$$

Make a smart choice for initializing your diagonal state prediction covariance matrix $P(0|0)$, as it will determine how fast your filter will converge depending on how close your initial guess for the state vector is to the real state vector. **Hint:** Use a fixed time step of $\Delta t = 0.01$ in your (Runge-Kutta) integrator and 'c2d' operations! **Hint2:** Note that the system dynamics equations $f(x, u, t)$ are trivial in the sense that they are fully described by the system input.

Appendix C: Description of Matlab Files

<F16traindata_CMabV_2025.mat>, <F16traindata_CMabV_2025.csv>

The F-16 aerodynamic dataset used in this year's multivariate spline assignment (CSV data for Python users).

<F16validationdata_2025.mat>, <F16validationdata_2025.csv>

The 'special validation dataset' used in this year's multivariate spline assignment, is used to calculate a final residual RMS value of your chosen models (CSV data for Python users).

<load_f16data2025.m>

Use this script in Matlab to load the F-16 data from disk.

<F16_PlotData.m>

This script shows how to interpret and plot the F-16 data in the data file.

<bsplinen_cart2bary.m>

This script transforms a global Cartesian coordinate to a local barycentric coordinate relative to a given simplex. Note that you don't need to use this function if you use the Matlab built-in TSEARCHN function.

<bsplinen_bary2cart.m>

This script transforms a local barycentric coordinate relative to a given simplex back to global Cartesian coordinates.